

Computer Vision Fundamentals [Week 1 Summary]

Dr. Patrick Mukala

CSCI435 - Computer Vision Algorithms and Systems

Why Computer Vision?

- Vision provides $\sim 80\%$ of human sensory input
- Many intelligent systems rely on visual understanding
- Core challenge: convert pixels into meaningful information

Goal: Enable machines to interpret and act on visual data.

Definition: Computer vision is the interdisciplinary field concerned with enabling machines to gain high-level understanding from digital images or videos.

Human Vision vs Machine Vision

Human Vision	Computer Vision
Context-aware and selective	Algorithm-driven and data-dependent
Robust to noise and ambiguity	Requires explicit modeling
Parallel processing	Sequential/parallel hybrid
Adaptive attention	Fixed receptive fields

Key Insight: CV approximates human vision, but optimizes for computation and reproducibility.

The Computer Vision Pipeline

- ➊ **Image acquisition** (sensor characteristics, calibration)
- ➋ **Preprocessing** (denoising, normalization, enhancement)
- ➌ **Feature extraction** (hand-crafted or learned)
- ➍ **Learning / inference** (model selection, optimization)
- ➎ **Decision/output** (uncertainty quantification)

Mathematical formulation:

$$\mathcal{I}(x, y) \xrightarrow{\phi(\cdot)} \mathbf{f} \xrightarrow{h(\cdot)} \hat{y}$$

where $\phi(\cdot)$ is the feature extractor and $h(\cdot)$ is the decision function.

Important: Each stage affects accuracy, efficiency, and robustness.

- **Low-Level Vision**

- Edge detection, filtering, optical flow
- Input: raw pixels; Output: primitive features

- **Mid-Level Vision**

- Segmentation, feature grouping, stereo matching
- Input: low-level features; Output: structured representations

- **High-Level Vision**

- Recognition, scene understanding, reasoning
- Input: mid-level representations; Output: semantic interpretations

Observation: Error propagation is a fundamental challenge — errors at early stages compound.

Core Vision Tasks

Task	Input	Output
Image Classification	Image	Class label
Object Detection	Image	Bounding boxes + labels
Semantic Segmentation	Image	Per-pixel class labels
Instance Segmentation	Image	Per-pixel instance labels
3D Reconstruction	Multiple images	3D model

These tasks form the foundation of modern computer vision systems.

$$f : \mathbb{R}^{H \times W \times C} \rightarrow \{1, 2, \dots, K\}$$

- Assign a single label to an entire image
- Uses extracted features or end-to-end deep learning
- Evaluation metric: Top-1/Top-5 accuracy

Limitation: No spatial localization — cannot answer "where is the object?"

Example datasets: ImageNet (1000 classes, 1.2M images), CIFAR-10/100

$$f(I) = \{(b_i, c_i, s_i)\}_{i=1}^N$$

where $b_i \in \mathbb{R}^4$ (bounding box), c_i (class), $s_i \in [0, 1]$ (confidence)

- Detect objects and their spatial locations
- Combines localization + classification
- Evaluation: mean Average Precision (mAP)

Challenge: Large search space — $\mathcal{O}(WH)$ possible windows

Semantic Segmentation:

$$f(I) \in \{1, \dots, K\}^{H \times W}$$

Instance Segmentation:

$$f(I) = \{(m_j, c_j)\}_{j=1}^M$$

- Pixel-level understanding
- Semantic vs instance vs panoptic segmentation

Applications: autonomous driving (road/vehicle/pedestrian delineation), medical imaging (tumor segmentation)

3D Reconstruction

- Recover 3D structure from 2D projections
- Uses epipolar geometry + multi-view constraints

Mathematical formulation:

$$\mathbf{x} = \mathbf{P}\mathbf{X} \quad (\text{projective transformation})$$

$$I_1, I_2, \dots, I_n \xrightarrow{\text{SFM/SLAM}} \text{3D model}$$

Applications: robotics navigation, AR/VR, digital heritage preservation

Search Space Explosion:

$2^{25} = 33,554,432$ possibilities for a 5×5 binary patch

Detection Complexity:

- Naive sliding window: $\mathcal{O}(WH \cdot N_{\text{scales}} \cdot N_{\text{aspects}})$
- For 1024×1024 image: $\sim 10^8$ windows

Implications:

- Need efficient features (e.g., integral images, cascade classifiers)
- Need efficient search (e.g., region proposals, anchor boxes)
- Need hardware acceleration (GPU, TPU, specialized chips)

Why Machine Learning?

- Hand-crafted rules do not scale to visual diversity
- Visual data is high-dimensional ($\sim 10^5$ dimensions for 224×224 image)
- Natural variation: illumination, viewpoint, occlusion, deformation

Mathematical formulation:

$$\mathbf{x} \xrightarrow{\text{feature mapping}} \phi(\mathbf{x}) \xrightarrow{\text{ML model}} \hat{y}$$

Goal: Learn patterns from data instead of explicit rule programming.

Core principle: All models are wrong, but some are useful (Box, 1976).

Supervised Learning

- Requires labeled data $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$
- Learns mapping: input $\rightarrow$ output
- Minimizes empirical risk: $\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}_i), y_i)$

Used for:

- Classification (discrete outputs)
- Regression (continuous outputs)
- Detection (structured outputs)

Challenge: Requires large, diverse, annotated datasets.

K-Nearest Neighbors (KNN)

$$\hat{y} = \arg \max_{y \in \mathcal{Y}} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{N}_K(\mathbf{x})} \mathbf{1}[y_i = y]$$

- Non-parametric, lazy learning
- No explicit training phase
- Used in image retrieval and baseline comparisons

Limitations:

- Inference: $\mathcal{O}(N \cdot d)$ — slow for large datasets
- Sensitive to irrelevant features
- Curse of dimensionality

Support Vector Machines (SVM)

Primal formulation:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i + b))$$

Dual (kernel) formulation:

$$f(\mathbf{x}) = \sum_{i=1}^N \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

- Finds maximum margin separating hyperplane
- Kernel trick enables non-linear decision boundaries
- Works well with engineered features (HOG, SIFT)

K-Means Clustering

Objective:

$$\min_{\{\mu_k\}_{k=1}^K} \sum_{i=1}^N \min_k \|\mathbf{x}_i - \mu_k\|^2$$

Iterative algorithm:

- ① Assignment: $c_i = \arg \min_k \|\mathbf{x}_i - \mu_k\|^2$
- ② Update: $\mu_k = \frac{1}{|\{i : c_i = k\}|} \sum_{i:c_i=k} \mathbf{x}_i$

Used for:

- Image segmentation (color clustering)
- Feature quantization (bag-of-visual-words)
- Unsupervised pre-training

Convolutional Neural Networks (CNNs):

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)} * \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$$

Key properties:

- Local connectivity (parameter efficiency)
- Weight sharing (translation equivariance)
- Hierarchical feature learning

Advantages:

- End-to-end learning from raw pixels
- State-of-the-art on all major benchmarks
- Transfer learning enables small-data scenarios

- 2500+ optimized computer vision algorithms
- Supports Python, C++, Java, GPU (CUDA/OpenCL)
- Real-time performance through SIMD optimization

Core modules:

- `cv2.imgproc`: filtering, geometric transforms, color conversions
- `cv2.feature2d`: SIFT, SURF, ORB feature extraction
- `cv2.dnn`: deep neural network inference
- `cv2.video`: optical flow, object tracking

Use case: Rapid prototyping → production deployment

Applications of Computer Vision

Domain	Applications
Autonomous vehicles	Lane detection, object tracking, depth estimation
Robotics	Visual SLAM, grasping, navigation
Healthcare	Medical image analysis, surgery assistance
Surveillance	Face recognition, anomaly detection
Manufacturing	Defect inspection, quality control
AR/VR	Pose estimation, scene reconstruction

Market size: \$48.6B by 2025 (CAGR 16.5%)

Key Takeaways

- Computer vision is a **pipeline** from pixels to semantic decisions
- Machine learning enables **scalability** and **robustness** to natural variation
- **Efficiency** is critical for real-time systems
- Trade-offs exist between accuracy, speed, and memory

Final Thought: Modern CV systems combine vision, learning, and systems thinking — understanding all three is essential for real-world deployment.

Fundamentals:

- Image processing (convolution, Fourier transforms, pyramids)
- Feature extraction (HOG, SIFT, SURF, ORB)

Advanced topics:

- Modern object detection (YOLO, Faster R-CNN, DETR)
- Video analysis (optical flow, action recognition)
- Building real CV systems with OpenCV

Resources:

- Szeliski, "Computer Vision: Algorithms and Applications"
- Goodfellow et al., "Deep Learning" (Ch. 9 on CNNs)
- OpenCV documentation and tutorials